HUMANIZE: JUDGEMENT ENGINEERING FOR AGENTIC CODING

Sihao Liu[1,2,*] **Ligeng Zhu**[1], **Zijian Zhang**[1,4], **Dongyun Zou**[3,5], **Zhengyang Zhang**[4]
Changye Li[4], **Song Bian**[1], **Song Han**[1,3], **Tony Nowatzki**[2]
[1]NVIDIA [2]UCLA [3]MIT [4]Tsinghua University [5]Nunchux AI

arXiv:2610.08900v2 [cs.AI] 8 Oct 2026

ABSTRACT

Agentic coding makes code generation cheap, but reliable completion remains difficult: the agent that writes the code is a weak judge of whether it is done. We present Humanize, a multi-agent orchestration workflow for agentic coding built around *judgement engineering*: explicit, mechanically enforced decisions at the boundaries between planning, implementation, review, and learning. A human approves a plan contract, a builder agent implements it in rounds, and a reviewer agent from another vendor decides completion; deterministic hooks, not a model, route work between these roles and enforce 72 mechanical gates. Viewed as a Markov chain over repository states, alternating builder and reviewer samples jointly from two models, so a defect survives only if both miss it. We study Humanize through its deployment, 118 public postmortems of real loops, and its applications. Over 68 versions in 108 days, it gathered 1,468 GitHub stars. Applications include a 567-file gem5 build-system migration under upstream review; Kernel Design Agents, which extend the loop with a kernel knowledge base and profiling feedback and placed in the top three of all three Full-Agent tracks of the MLSys 2026 FlashInfer contest; and, through Humanize Olympiad Agents (HOA), full scores in IOI 2026, IMO 2026, IPhO 2026, and IBO 2024, 418.5/437 in IChO 2026 (gold-medal). Humanize also achieves 672/672 on PutnamBench and ranks first (251/303) on Lean-Eval's leaderboard even competing with professional mathematicians. The postmortems show that independent review catches unsupported builder claims, but stopping remains a key weakness. In reports that separate rounds by phase, two thirds of rounds occurred after implementation was accepted. This evidence is observational, not a controlled comparison of workflows.

Code:[1] https://github.com/PolyArch/humanize

1 INTRODUCTION

Agentic coding makes writing code cheap. Instead of stopping after a single prompt, agents such as Claude Code and Codex CLI pursue a target through cycles of planning, implementation, review, and revision (Anthropic, 2025a; OpenAI, 2025). Errors accumulate as the implementation grows, and the agent that wrote the code is a weak judge of it: language models struggle to correct their own reasoning without external feedback (Huang et al., 2024), favor their own outputs (Panickssery et al., 2024), and can reward-hack by changing tests (Von Arx et al., 2025). The challenge shifts from producing code to verifying correctness, goal alignment, and completion.

Existing designs leave the completion decision with the agents that do the work. Self-refinement loops let a model revise its own output (Shinn et al., 2023; Madaan et al., 2023), task verification is a major failure category of multi-agent systems (Cemri et al., 2025), and the Ralph loop re-feeds a coding agent the same prompt until the agent itself declares the task done (Huntley, 2025; Anthropic, 2025d). Reliable completion instead requires explicit decisions about the plan, the builder's claims, the review, and when to stop.

[*]Work done while at UCLA.

[1]Meanwhile, Humanize2 is in active development at https://github.com/humanfia/humanize.

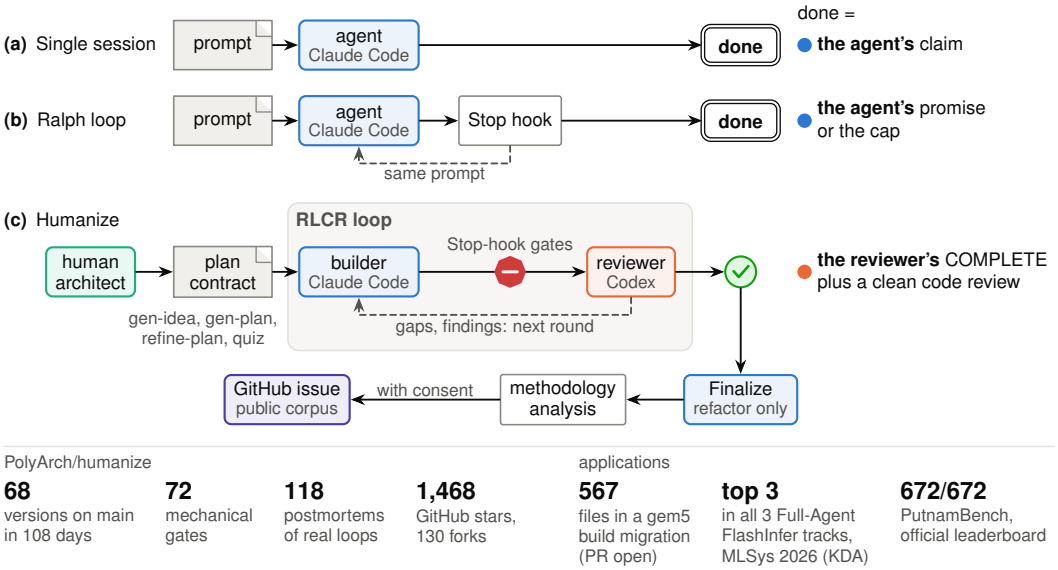


Figure 1: **Humanize separates building from acceptance through independent review and mechanical gates.** (a)–(c) The right column compares who declares completion in a single session, a Ralph loop (Huntley, 2025), and Humanize. In (c), Stop-hook gates and an independent reviewer check each builder round; dashed arrows return work. The bottom strip summarizes the project's development record and applications.

We present Humanize, a multi-agent orchestration workflow built around *judgement engineering*: explicit, mechanically enforced decisions at the boundaries between planning, implementation, review, and learning (Figure 1). The human approves a plan contract that fixes the goal and its acceptance criteria. A builder agent implements the plan in rounds and summarizes its claims. A reviewer agent from another vendor evaluates alignment and correctness and alone can declare the work complete. Deterministic hooks, not a model, route work between these roles and enforce mechanical gates, so the roles hold when an agent errs or over-claims. Viewed as a Markov chain over repository states, the alternation of builder and reviewer samples jointly from two models, and a defect survives only if both miss it.

We study Humanize, a Claude Code plugin whose core loop is RLCR (Ralph-Loop with Codex Review). Our contributions are:

- **Design.** Judgement engineering for orchestrating coding agents, built on alternating two models as a joint sampler and realized in a plan contract, a round protocol between builder and reviewer, and 72 mechanical gates (§2 and §3).
- **Field evidence.** A study of the deployment of Humanize and of 118 public postmortems of real loops, most written by its own methodology analysis (§4).
- **Applications and lessons.** Applications to a mapper optimization, a 567-file build migration, contest GPU kernels from a loop extended with a kernel knowledge base and profiling feedback, PutnamBench, and olympiads, which show that independent judgement can come from a second model or from an oracle (§5), and five design lessons (§6).

The evidence is observational: it shows what the design enables and where it fails, not how it compares with other workflows under controlled conditions.

2 Judgement Engineering

Humanize starts from one premise: **do not automate a plan the human cannot judge.** When one agent plans, edits code, and declares completion in a single conversation, the human must reconstruct the goal, the changes, and the supporting evidence from a long transcript or diff, and verification waits until both the implementation and its review context have grown. *Judgement engineering* instead

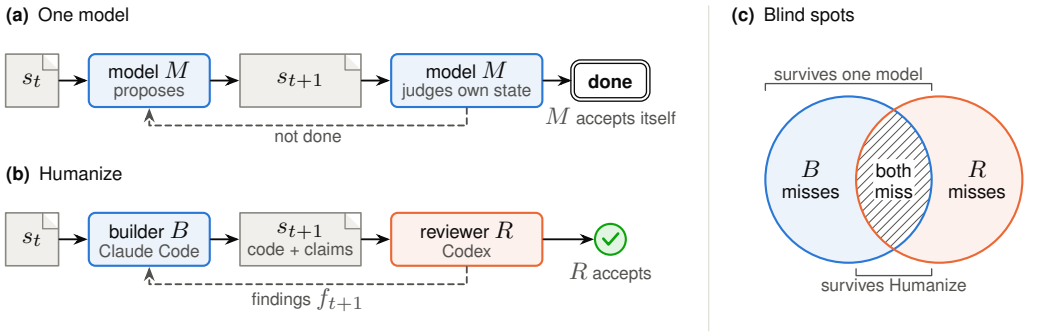


Figure 2: **Alternating models reduces accepted defects when their blind spots differ.** (a) One model M proposes and judges each state, leaving its blind spots unchecked. (b) Humanize forms a joint sampler: builder B proposes each state, and reviewer R from another vendor judges it and decides acceptance. (c) A defect overlooked by B survives the loop if R also misses it.

places explicit, mechanically enforced decisions at the boundaries between planning, implementation, review, and learning. Four principles guide the design.

- **Judge the plan before the code.** A plan contract fixes the goal, acceptance criteria, scope, and tasks before execution, and gives the reviewer a stable basis for judgement.
- **Separate building from judging.** The builder ends each round with a summary that links its claims to evidence and states what remains open; a different model decides whether the claims hold, and only it can declare completion (§2.1).
- **Decompose the completion decision.** Alignment review checks the claim against the plan and separates blockers from side issues that can be queued; code review checks the diff for defects. A change can follow the plan yet contain defects, or be locally correct yet miss the goal.
- **Measure progress by evidence, not rounds.** A stalled loop should revisit the scope, escalate, or stop; a finished loop should leave lessons for later tasks.

These principles generalize the evaluator-optimizer pattern (Anthropic, 2024) from single responses to whole working sessions, and Humanize keeps the orchestration itself in deterministic code rather than in a model.

2.1 Alternating Models as Joint Sampling

We view a coding session as a Markov chain over states s_t, the repository together with the plan and the review record after round t. A single agent draws every step from one model, $s_{t+1} \sim P_M(\cdot \mid s_t)$, and stops when the same model accepts its own state. Proposal and acceptance then share one set of blind spots: whatever the model systematically gets wrong, it also tends to accept (Huang et al., 2024; Panickssery et al., 2024).

Humanize alternates two models instead (Figure 2). The builder B proposes the next state given the reviewer's last findings, $s_{t+1} \sim P_B(\cdot \mid s_t, f_t)$, and a reviewer R from another vendor, in a fresh context, returns findings $f_{t+1} \sim P_R(\cdot \mid s_{t+1})$ that either accept the state or condition the next proposal. Like a two-block Gibbs sampler, the chain alternates between two conditionals, here supplied by two different models, so every step is shaped by both, and the chain can stop only in a state that R accepts. Let b be the probability that the builder introduces a defect and overlooks it, and m_R the probability that the reviewer misses it given that the builder did. The defect survives a single-model loop with probability b and the alternation with probability $b\,m_R$. Because m_R approaches one as the reviewer's blind spots approach the builder's, the joint chain accepts fewer defective states whenever the two models' blind spots differ; this is why Humanize pairs Claude Code as builder with Codex as reviewer by default. An external oracle, such as a proof checker or a test harness, adds a third factor, its own miss rate m_O, and where it is complete for a claim it can stand in for the reviewer's independence (§5). The price of alternation is that the reviewer's false positives also enter the chain, as extra rounds (§6).

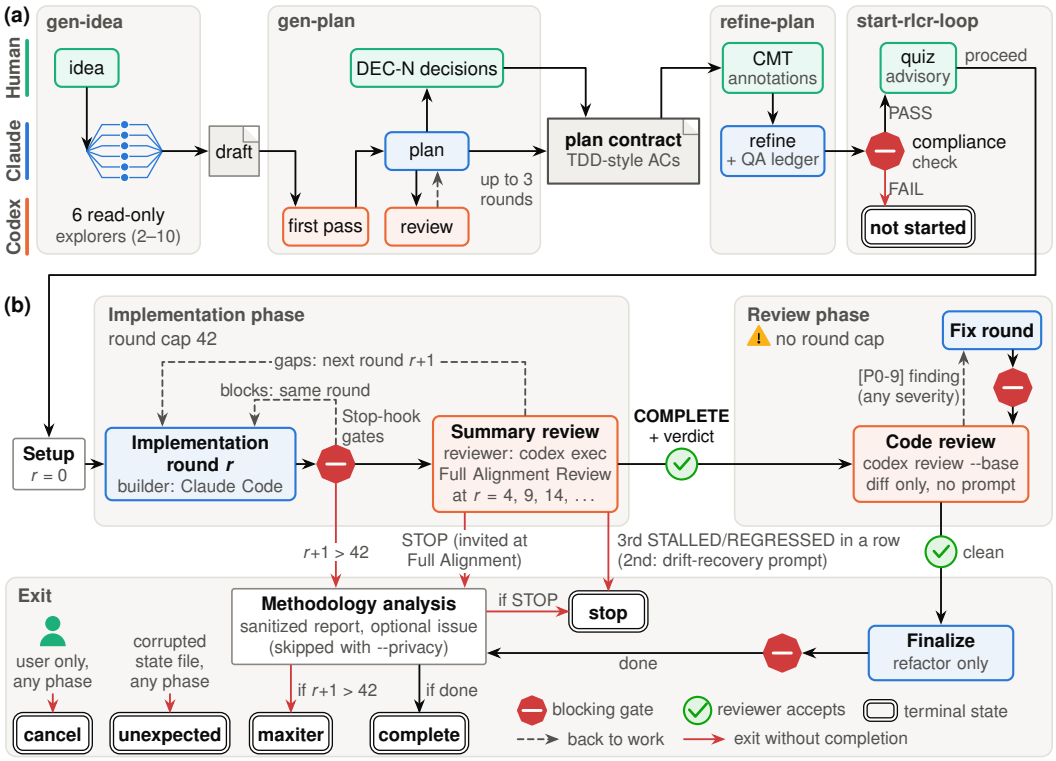


Figure 3: **Humanize fixes the plan before execution and makes completion a reviewer decision.** (a) The planning chain separates human, Claude, and Codex roles. Lanes denote actors, regions denote commands, and the first three commands are optional. (b) The RLCR state machine shows default transitions and explicit exits. Octagons mark blocking gates; dashed arrows return work; red arrows exit without completion.

3 System Design

Humanize implements the workflow as a Claude Code plugin (Anthropic, 2025c). It provides five slash commands, four subagents, six skills, and hooks for four Claude Code events (Anthropic, 2025b). We describe the plugin's last release (Figure 3).

3.1 Planning as a Contract

Humanize separates planning from implementation. `gen-idea` expands a rough idea into a draft using parallel, read-only subagents that explore different directions. `gen-plan` turns this draft into a structured plan, with coding forbidden during plan generation. Claude and Codex draft and critique the plan for up to three rounds, then route unresolved points to the human. The plan is a contract, not a disposable prompt: it defines the goal, acceptance criteria with positive and negative tests, path boundaries, and tasks assigned to Claude for coding or Codex for analysis.

Before execution, the plugin runs two checks. A plan-compliance check rejects unrelated plans and branch-switching instructions that would break loop accounting. An advisory quiz asks the human two multiple-choice questions about the plan's mechanism and architecture. This added friction helps expose plans accepted without being understood.

3.2 The Round Protocol

The builder works in one Claude Code session, with rounds defined as "One round = the agent believes the entire plan is finished." The Stop hook mediates each exchange with the reviewer (Figure 4). It supplies the plan, the goal tracker, and the previous review findings. The goal tracker's immutable section holds the goal and acceptance criteria. The hook also points the builder to BitLesson, whose

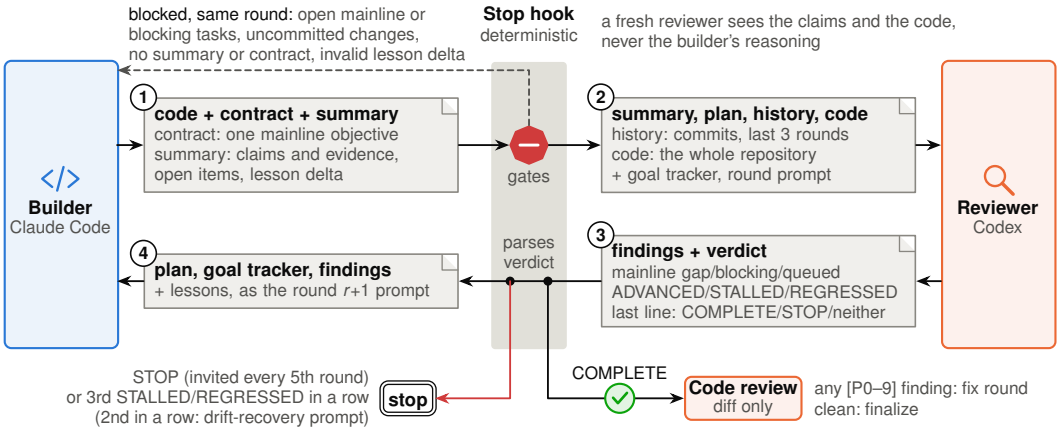


Figure 4: **Humanize's Stop hook makes each RLCR round an evidence-based builder–reviewer exchange.** (1) The builder submits code, a round contract, and a claim summary. (2) After the hook's gates pass, a fresh reviewer checks the claims against the plan, history, and code. (3) The reviewer returns findings and a verdict. (4) The hook uses them to prompt the next round, start code review, or stop the loop.

selector chooses project lessons relevant to each task. The builder returns commits, a round contract naming one mainline objective, and a summary of changes, supporting evidence, and open work. The contract queues side issues so they do not take over the round.

When the builder tries to stop, the hook checks these artifacts before launching a fresh Codex process. The reviewer reads the plan, recent rounds, code, and the builder's claims and evidence. It separates findings into mainline gaps, blocking side issues, and queued side issues, and reports whether the mainline advanced, stalled, or regressed. Returning `COMPLETE` requires a finished plan; deferred tasks count as incomplete. The hook parses the verdict and turns the findings into the next round's prompt. Two consecutive stalled rounds trigger a drift-recovery prompt. Every fifth round is a Full Alignment Review that audits all acceptance criteria. Invitations to stop the loop are restricted to these reviews.

This exchange implements the alternation in §2.1. The reviewer judges the repository state and the builder's claims with evidence, not the builder's reasoning. The builder receives findings through a prompt it cannot edit, and the hook parses the verdict.

Once the reviewer accepts the implementation, the loop enters code review. Codex checks the diff against the base commit and emits severity-tagged findings. Any finding starts a fix round within this phase; a clean review allows finalization through a behavior-preserving simplification pass. This phase has no round cap; §6 examines its cost.

The loop has five explicit terminal states: `complete`; `stop`, from the reviewer's verdict or three consecutive stalled or regressed rounds; `maxiter`, at the implementation cap (42 rounds by default); `cancel`, which requires a user request; and `unexpected`, for a corrupted state file. After `complete`, `maxiter`, or a reviewer-requested `stop`, an analysis agent turns the recorded rounds into a sanitized methodology report. The user may file it as an issue after two confirmations. These reports form the corpus in §4.

3.3 Mechanical Gates

Humanize enforces the workflow through code rather than relying on prompts. We catalogued 72 mechanical gates (Appendix A). Hook validators block builder writes to the state file, the plan copy, the immutable goal, and other rounds' files. Before launching a review, the Stop hook checks that summaries, lesson deltas, task lists, and commits are present. It also checks that review outputs are well formed. These gates enforce artifact presence and form; the reviewer judges content. They keep the workflow mandatory when an agent errs or over-claims. Enforcement grew with the system: the Stop hook expanded from 975 to 2,221 lines.

Runtimes. In the default runtime, the user's Claude Code session builds, and Codex reviews with `gpt-5.5` at high effort. The plugin protects independence through a reviewer from another vendor, a fresh process per review, hook-parsed verdicts, and prompts the builder cannot edit. Community-contributed Codex and Kimi runtimes implement fewer of these protections. Host-tool dependencies also affect reliability: later Codex CLI releases broke parts of the plugin after its last release.

4 Deployment and Postmortem Study

Deployment. Humanize grew out of GAAC, an earlier Claude Code methodology plugin (Liu, 2025). Its first release contained the core loop. Over 108 days, 68 versions reached the main branch, adding code review, agent teams, the Codex and Kimi runtimes, BitLesson, and joint plan generation by Claude and Codex. The last release added round contracts, drift control, and the methodology analysis that generated most of the reports below. Development then moved to a dev branch and a standalone successor (Humanfia, 2026b).

Adoption continued after the plugin's last release. Its repository had 1,468 stars, with 61% gained after that release, 130 forks, and pull requests from 34 authors. Contributors outside the core team supplied the Codex and Kimi runtimes, BitLesson, plan refinement, round contracts, and a monitoring dashboard.

Postmortem corpus. We analyze Humanize's public issue tracker as an observational corpus of methodology reports. It contained 150 issues from 52 authors, including 118 postmortems of real loops. Of these postmortems, 113 carry the signature or template of the methodology analysis (§3.2).

LLM agents coded every issue using a written codebook (Appendix B). The codebook records issue type, stated round count and exit, reported pressures, and proposed mechanisms. Agreement with a blind second LLM coder reached Cohen's $\kappa = 0.93$ for round counts and $\kappa = 0.79$ for primary pressure. Artifact state, review tails, scope, and convergence were the most frequent pressures, each present in 64–73% of postmortems. Review tails and convergence were also the most common primary pressures.

The corpus is not a representative sample of all runs. Users chose what to file, four reporters wrote half the reports, and the methodology analysis runs for selected exits. We use these reports to examine where judgement boundaries held, where they were too weak, and when further review cost more than it helped.

Loop length. Of the 93 postmortems with a stated round count, the median loop ran 10 rounds and the longest ran 87. In the 27 postmortems that separated rounds by phase, review dominated: 364 of 541 rounds (67.3%) occurred in the uncapped code-review phase, after implementation was accepted. Before acceptance, stagnation stops followed the review schedule: every such stop with a stated round number fell on a Full Alignment Review, where the reviewer is invited to stop the loop.

5 Applications

We next examine applications with checkable outcomes. They are application reports, not controlled comparisons.

Loom: faster search with trace equivalence[2]. Loom's placement-and-routing mapper uses simulated annealing, so optimizing its search loop can change the decisions it makes (Liu, 2026). The optimization contract required exact agreement with the reference mapper's per-level decision trace. Under a 180-second budget, the campaign raised the explored proposal slots from 97,138 to 227,777 (2.34×), and four faster candidates that broke trace equivalence were reverted. This is a throughput result, not evidence of better mappings.

[2] https://github.com/PolyArch/loom

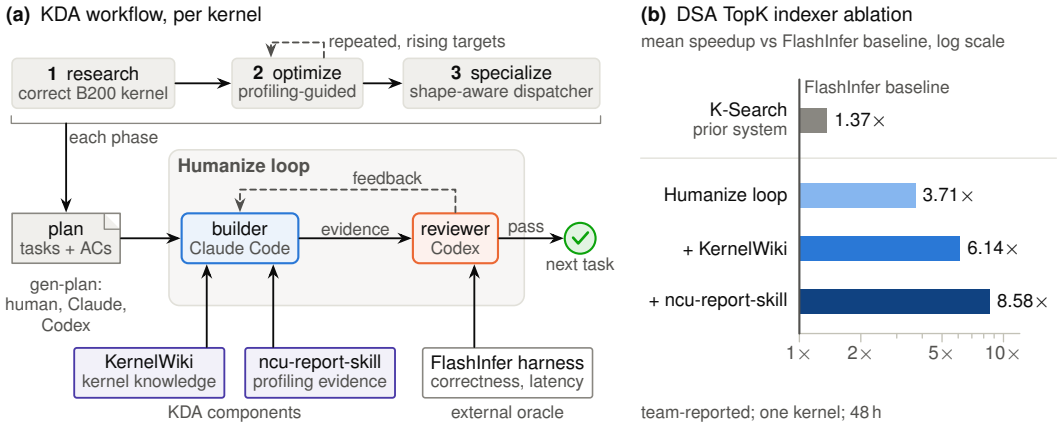


Figure 5: **Kernel knowledge and profiling feedback improve KDA's reported DSA indexer speedup.** (a) The builder queries KernelWiki for production kernels, contest submissions, and documentation, and uses ncu-report-skill to interpret Nsight Compute profiles. The reviewer checks this evidence against each acceptance criterion, using the official FlashInfer harness as oracle. (b) The team-reported 48-hour DSA TopK indexer ablation adds each component to the Humanize loop. K-Search is a separate prior system.

Gem5: a build-system migration[3]**.** Replacing gem5's SCons build must not change what the build generates. A loop on the Humanize plugin, with Claude building and Codex reviewing, replaced it with CMake and Ninja and added a Bazel overlay across 567 files in 133 commits. The first author validated the result with the full build, unit tests, and matching simulation objects across the two new builds; upstream, its CI has not passed and the pull request awaits maintainer review. An agent loop can produce a change of this size; accepting it remains a human decision.

Kernel Design Agents (KDA)[4]**.** KDA builds a domain-specific knowledge base and fast profiling feedback around the Humanize loop (HAN Lab Kernel Mafia, 2026) (Figure 5). KernelWiki is a retrieval store of production kernel pull requests, contest submissions, and documentation that the builder queries while it designs and optimizes a kernel; the ncu-report-skill turns Nsight Compute profiles into evidence that the builder acts on and the reviewer checks against each acceptance criterion. Each kernel passes through three plans, for a correct kernel, for profiling-guided optimization with rising speedup targets, and for shape-aware dispatch, and an RLCR loop executes each plan with Claude as coding agent and Codex as acceptance agent. In the MLSys 2026 FlashInfer contest (FlashInfer, 2026), KDA placed first in MoE, second in DSA, and third in GDN in the Full-Agent division, with team-reported mean speedups over the FlashInfer baselines of 19.08× for the DSA indexer, 4.54× for DSA attention, and 1.92× for GDN prefill; two of its five kernels remained slower than the baseline. A later team-reported ablation on the DSA indexer, under a 48-hour budget, credits each part: the loop alone reached 3.71×, KernelWiki raised it to 6.14×, and the profiling skill to 8.58×, against 1.37× for K-Search (Cao et al., 2026). In the same rerun, KDA exceeded K-Search on all three tracks: 11.9× against 3.76× for DSA, 1.16× against 1.04× for GDN, and 0.72× against 0.27× for MoE, in mean speedup over the FlashInfer baselines.

Humanize Olympiad Agents (HOA)[5]**.** Humanfia's HOA applies the builder-reviewer workflow to olympiad and benchmark problems (Humanfia, 2026a), with a completion check for each discipline. Two runs proved all six IMO 2026 problems in Lean from AxiomProver's formal statements (Axiom Math, 2026), one with GPT-5.6 Sol in both roles and one with Kimi K3 building and Codex reviewing. All six IOI 2026 tasks received full scores from the Codeforces judge. Both IPhO 2026 theory runs scored 30/30 against the official marking scheme, IChO 2026 reached 418.5/437 theory points with all 68 formalization targets covered. The IBO 2024 run matched all 100 official answers. We use 2024 because the committee keeps problems confidential for two years. These scores are self-graded;

[3] https://github.com/gem5/gem5/pull/2969
[4] https://nvlabs.github.io/kda
[5] https://github.com/humanfia/hoa

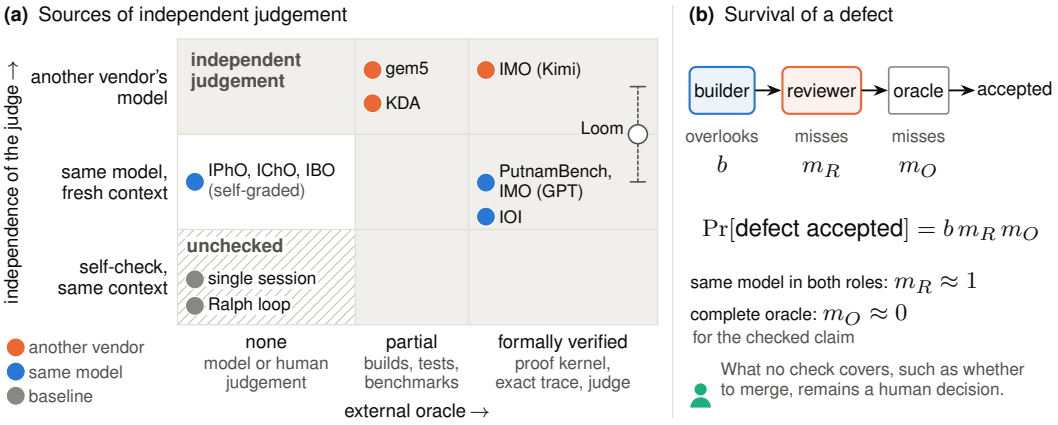


Figure 6: **Independent judgement can come from a second model or an external oracle.** (a) Applications and baselines are positioned by reviewer independence and oracle strength; acceptance is unchecked when both are absent. (b) A defect is accepted with probability $b\, m_R\, m_O$. A reviewer from another vendor can lower m_R; a complete oracle drives m_O toward zero for the claim it checks.

IBO agents could see the key. The releases share specialized agent flows with a judgement boundary: preserve questions, check artifacts, and identify who accepted them.

PutnamBench[6] and Lean-Eval[7]. For formal proofs, the Lean kernel and a statement-preservation check run before the reviewer, which must also obtain an independent proof verifier's approval. Humanize achieves 672/672 on PutnamBench (Humanfia, 2026c). Lean-Eval requires submitted formal proofs to pass its Comparator check, and Humanize ranks first on its legacy leaderboard, ahead of professional teams including AxiomProver (Axiom Math), Aleph Prover (Logical Intelligence), Aristotle (Harmonic), and Seed Prover (ByteDance) (Lean FRO, 2026).

Two sources of independent judgement. The applications differ in where their independent judgement comes from (Figure 6). Where external checks are partial, as the builds and tests of the gem5 migration or the contest harness of KDA, a reviewer from another vendor carries the judgement that the checks leave open, and the gem5 change still awaits a human decision. Where an oracle is complete for the claim, as the Lean kernel for PutnamBench and IMO or the judge for IOI, one model in both roles sufficed, because the oracle drives m_O (§2.1) toward zero for the property it checks. Results with neither source are the weakest: the natural-language olympiad scores come from HOA's own grading and are estimates, and the single-session agent and the Ralph loop sit in this unchecked corner by construction. The applications show what such loops can reach, not how much the reviewer adds: no public comparison runs the same builder with and without the reviewer under the same budget, and within a run the loop often confirms a first attempt, as in the PutnamBench rerun where 86 of 98 problems passed at the first turn.

6 Findings

Together, the postmortems and the applications yield five lessons for orchestrating coding agents.

Independent review is trusted but expensive. The most consistent lesson is that the reviewer, not the builder, should decide completion. Forty-five of the 118 postmortems describe builders claiming more than they had done, and KDA's team reports the same failure in kernel work: its builder once used its own first kernel as the baseline and declared the task done. Few reports fault the reviewer's verdicts; the complaints concern how review is paced. A one-finding-per-round cadence turns valid review into a long tail, and the code reviewer, given only the diff and no history, rediscovers related defects one path at a time. Review should therefore be batched, not weakened: group related findings and give the reviewer memory of earlier rounds, without importing the builder's reasoning.

[6] https://trishullab.github.io/PutnamBench/leaderboard
[7] https://lean-lang.org/eval/legacy

Iteration does not guarantee convergence. The review phase was exempt from the round cap; in the postmortems that split phases, two thirds of rounds came after acceptance (§4); one ran 87 rounds against a cap of 42. A mainline verdict such as "advanced" is useful only if it is tied to measurable movement toward the acceptance criteria, and one report recorded "advanced" in every round of a loop that ended with none of its eight criteria fully met. Termination must be designed: every phase needs a bound, a stop rule should follow the state of the work rather than a fixed schedule, and the loop needs an exit for work that is blocked rather than failing.

Plans and scope are judgement boundaries. Large plans create too many simultaneous obligations, vague acceptance criteria produce ambiguous stopping conditions, and reviewers discover real issues whose relation to the accepted plan is unclear. Builder and reviewer must also read the contract the same way: the plugin's rule that deferred work counts as incomplete clashed with plans that deliberately left work for later. Discoveries should be routed rather than ignored: a blocking issue should explain why it prevents the accepted goal, a queued issue should be recorded without extending the loop, and a plan amendment should be explicit. Enforcement presupposes a plan that the human can judge.

Evidence and environment decide termination. Several sessions stalled not because the implementation was wrong but because evidence could not be collected consistently: acceptance criteria depended on unavailable infrastructure, or the builder's and reviewer's environments disagreed. Verification is therefore part of the judgement design, not a test-running detail. The strongest application results rest on oracles outside both agents, which can replace a second model as the independent judge (§5). A robust loop records its environment assumptions and asks the human for a decision instead of spending rounds on evidence it cannot collect.

The loop needs judgement around it. The methodology analysis made reports cheap: the 118 postmortems contain 771 numbered suggestions. No postmortem proposal is strongly linked to a change on the plugin's main branch, whereas concrete bug reports were fixed within days; 86 of the 150 issues arrived after the plugin's last release, when development had moved to the dev branch and a standalone successor. Self-generated feedback scales reporting more easily than learning: a loop that judges code also needs a process that judges its own reports.

7 Related Work

Coding agents and loops. Tool-using agents interleave reasoning and action (Yao et al., 2023), operate on whole repositories (Yang et al., 2024; Wang et al., 2025), and refine their output with their own critique or with execution feedback (Shinn et al., 2023; Madaan et al., 2023; Chen et al., 2024). The Ralph loop reruns such an agent on the same prompt until it emits a completion promise (Huntley, 2025; Anthropic, 2025d); Humanize keeps the outer loop but gives the stop decision to an independent reviewer, and it adds an enforced review to the explicit plans of spec-driven tools (Delimarsky, 2025; Swaminathan & Singh, 2025). Vendors have since released native loops, such as Claude Code's `/goal` with a separate evaluator (Anthropic, 2026), and loop design has become a research topic (Li et al., 2026; Lulla et al., 2026); none of these studies evaluates Humanize.

Multi-agent orchestration. Multi-agent frameworks organize models into software roles through standard operating procedures, chat chains, or programmable conversations (Hong et al., 2024; Qian et al., 2024; Wu et al., 2024). A taxonomy of their failures groups them into system design, inter-agent misalignment, and task verification (Cemri et al., 2025). Humanize is deliberately small: two model roles and a human, with routing held by deterministic hooks rather than by a model, and with verification assigned to a dedicated role.

Models as judges. Separate verifiers and critics catch errors that generators miss (Cobbe et al., 2021; McAleese et al., 2024), and models are widely used to judge model output (Zheng et al., 2023; Zhuge et al., 2025). Critics also report bugs that do not exist (McAleese et al., 2024), and cross-model code review is asymmetric: in a controlled study, Codex reviewing Claude's drafts, Humanize's default pairing, was counterproductive when the reviewer could not run tests (Xiang et al., 2026). Humanize's reviewer is an agent that can execute code and judges whole sessions; the review tails of §4 are one cost of that design.

8 LIMITATIONS

Humanize depends on other vendors' command-line tools. Its orchestration runs on Claude Code's hook interface and its reviewer on Codex CLI, whose releases after the plugin's last release broke parts of it, and only the Claude Code plugin installs the full set of independence protections (§3.3). Its termination rules are coarse: the round cap binds only the implementation phase, the reviewer is invited to stop the loop only every fifth round, and no exit separates work blocked by its environment from work that fails (§6). Its gates check the presence and form of artifacts rather than their content, so a well-formed but false summary must be caught by the reviewer, and the tool-call validators fail open on a malformed state file (Appendix A). The code reviewer sees only the diff, without memory of earlier rounds, and Humanize keeps no telemetry on gate firings or round costs. Finally, Humanize has not been evaluated in a controlled comparison: the evidence in this paper is observational, and no public study runs the same builder with and without its reviewer.

9 CONCLUSION

Humanize treats judgement as the primary design problem of agentic coding. Its contribution is not a larger agent organization or a stronger claim about autonomous coding, but a compact multi-agent workflow that alternates two models as builder and reviewer, makes goals, claims, reviews, evidence, and stopping decisions explicit, and enforces them mechanically. In deployment, independent review caught over-claiming builders, and the builder-reviewer pattern supported applications from a build-system migration to a complete PutnamBench proof set, but iteration did not guarantee convergence, and cheap reports still needed judgement to yield changes. Reliable agentic coding requires not just generation but engineered judgement around it. Development continues in a standalone successor, which moves the loop out of host-CLI hooks into its own runtime (Humanfia, 2026b); we will describe it separately.

ACKNOWLEDGMENTS

We thank the contributors to the first major version of Humanize for the following:

- Codex and Kimi runtimes: Yaoyang Zhou.
- Plan convergence, bitLesson, and plan refinement: Dongyu Zhang.
- Round contracts and drift control: Yan Xu.
- Integral review context and dashboard: Chao Liu.
- Idea generation: Shinan Liu.
- Roadmap discussions and infrastructure support: Ceyu Xu.
- SGLang integration and infrastructure discussion: BBuf Xiaoyu Zhang.
- HAN Lab Kernel Mafia on the KDA contest: Dongyun Zou, Zhekai Zhang, Shang Yang, Hao Kang, Muyang Li, Han Cai, and Ligeng Zhu.
- Olympiad Competition, PutnamBench and Lean-Eval Leaderboard: Zhengyang Zhang, Jing Xiong, Hongzhou Lin, Jui-Hui Chung, and Ligeng Zhu.
- FlashInfer team: Yixin Dong, Hongyi Jin, Zihao Ye, Avery Huang, and Junru Shao,
- The MLSys kernel contest organizers, the PutnamBench and Lean-Eval maintainers.

also including everyone who opened a pull request or filed an issue or shared an idea during discussion. The main development is led by NVIDIA Research, while we appreciate the development and compute support from the UCLA PolyArch, MIT HAN Lab, and CMU Catalyst.

REFERENCES

Anthropic. Building effective agents. Anthropic Engineering blog, 2024. URL https://www.anthropic.com/engineering/building-effective-agents.

Anthropic. Claude 3.7 Sonnet and Claude Code. Anthropic announcement, 2025a. URL https://www.anthropic.com/news/claude-3-7-sonnet.

Anthropic. Hooks reference: Claude Code docs. Claude Code documentation, 2025b. URL https://code.claude.com/docs/en/hooks.

Anthropic. Customize Claude Code with plugins. Claude blog, 2025c. URL https://claude.com/blog/claude-code-plugins.

Anthropic. Ralph Wiggum plugin for Claude Code. anthropics/claude-code repository, plugins/ralph-wiggum, 2025d. URL https://github.com/anthropics/claude-code/tree/main/plugins/ralph-wiggum.

Anthropic. Keep Claude working toward a goal. Claude Code documentation, 2026. URL https://code.claude.com/docs/en/goal.

Axiom Math. AxiomProver at IMO 2026. GitHub repository AxiomMath/IMO2026, 2026. URL https://github.com/AxiomMath/IMO2026.

Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model, 2026. URL https://arxiv.org/abs/2602.19128.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 38, 2025. doi: 10.52202/085713-4082. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b1041e52d3be19f0a9bc491657488e4a-Abstract-Datasets_and_Benchmarks_Track.html.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to Self-Debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=KuPixIqPiq.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Den Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2025. URL https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.

FlashInfer. MLSys 2026 FlashInfer AI kernel generation contest (NVIDIA track). Competition website, 2026. URL https://mlsys26.flashinfer.ai/.

HAN Lab Kernel Mafia. Developing GPU kernels with agentic loops. Technical report, MIT HAN Lab, 2026. URL https://github.com/mit-han-lab/mlsys2026-flashinfer-contest/blob/main/docs/HAN_Lab_Kernel_Mafia_Technical_Report.pdf. MLSys 2026 FlashInfer contest report.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=IkmD3fKBPQ.

Humanfia. HOA: Humanize Olympiad Agents. Software repository (CITATION.cff), 2026a. URL https://github.com/humanfia/hoa.

Humanfia. Humanize (hmz): The agent flow system. GitHub repository humanfia/humanize, 2026b. URL https://github.com/humanfia/humanize.

Humanfia. Humanfia at PutnamBench: 672/672 verified proofs. Public repository and proof verification pipeline, 2026c. URL https://github.com/humanfia/hoa.

Geoffrey Huntley. Ralph Wiggum as a "software engineer". Blog post, 2025. URL https://ghuntley.com/ralph/.

Lean FRO. Lean-Eval: Lean AI formalization leaderboard. Benchmark repository and leaderboard (leanprover/lean-eval, leanprover/lean-eval-leaderboard), 2026. URL https://lean-lang.org/eval/legacy/.

Han Li, Zhemin Fang, Rili Feng, Yingqi Zhao, Jiaheng Liu, Pengfei Gao, He Ye, Dayi Lin, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. LoopsBench: From harness engineering to loop engineering in coding agent evaluation, 2026. URL https://arxiv.org/abs/2608.00267.

Sihao Liu. GAAC: GitHub-as-a-context. GitHub repository SihaoLiu/gaac (Claude Code methodology plugin), 2025. URL https://github.com/SihaoLiu/gaac.

Sihao Liu. *Loom: Full-Stack Compilation and Automated Exploration of Heterogeneous Multi-Core Spatial Accelerator Systems*. PhD thesis, University of California, Los Angeles, 2026. URL https://escholarship.org/uc/item/84k2g515.

Jai Lal Lulla, Vahram Nersesyan, Seyedmoein Mohsenimofidi, Christoph Treude, and Sebastian Baltes. Loop engineering: Building blocks, adoption, and impact, 2026. URL https://arxiv.org/abs/2608.21884.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594, 2023. doi: 10.52202/075280-2019. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.

Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. LLM critics help catch LLM bugs, 2024. URL https://arxiv.org/abs/2407.00215.

OpenAI. Codex CLI. Software repository, 2025. URL https://github.com/openai/codex.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, pp. 68772–68802, 2024. doi: 10.52202/079017-2197. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/7f1f0218e45f5414c79c0679633e47bc-Abstract-Conference.html.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024. doi: 10.18653/v1/2024.acl-long.810. URL https://doi.org/10.18653/v1/2024.acl-long.810.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652, 2023. doi: 10.52202/075280-0377. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

Nikhil Swaminathan and Deepak Singh. Introducing Kiro. Kiro blog, 2025. URL https://kiro.dev/blog/introducing-kiro/.

Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR blog, 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=OJd3ayDDoF.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *Conference on Language Modeling (COLM)*, 2024. URL https://openreview.net/forum?id=BAakY1hNKS.

Zuodong Xiang, Yike Zhang, YueMing Zhang, and Hailu Xu. Cross-Model LLM code review: Should you use Claude to review Codex or vice versa?, 2026. URL https://arxiv.org/abs/2607.21656. Accepted to Agentic SE workshop at KDD 2026 (arXiv comment).

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pp. 50528–50652, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pp. 46595–46623, 2023. doi: 10.52202/075280-2020. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-Judge: Evaluate agents with agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 80569–80611, 2025. URL https://proceedings.mlr.press/v267/zhuge25a.html.

A Mechanical Gates

We count as a gate every point where the code of Humanize checks the loop and can block, end, or redirect it; Table 1 groups the 72 gates by category. Twenty-nine live in the tool-call and prompt validators, which only the Claude Code plugin installs, and 32 run in the Stop hook. One gate fails open: on a malformed state file, the tool-call validators exit with code 1, which Claude Code treats as a non-blocking error (Anthropic, 2025b), so the call proceeds.

Table 1: **The largest gate categories protect state, plans, and round isolation.** All 72 gates of Humanize by category.

Category	Gates	What the gates enforce
State integrity	12	read-only loop state; schema, branch, and corruption checks
Plan and scope protection	11	plan and compliance checks at setup; frozen plan copy and goal
Round isolation	10	each round writes only its own files; other rounds are unreadable
Git hygiene	9	a clean tree at setup and at every stop; loop state is never committed
Review protocol	7	valid review and verdict each round; STOP and drift breakers; round cap
Session hygiene	6	a locked-down analysis phase; no direct hook runs; only the user cancels
Artifact presence	5	an initialized goal tracker; a contract and a summary before each review
Lesson memory	5	every summary records a valid BitLesson delta
Environment precondition	4	Codex, git, and a base branch are available
Completion discipline	3	no open mainline or blocking tasks; no changed file over 2,000 lines

B Corpus Coding

The corpus holds the 150 issues of PolyArch/humanize: 118 postmortems and 32 bug reports, feature requests, questions, and other issues. For each postmortem, LLM agents coded the stated round count (93 postmortems), the exit (complete 37, stagnation stop 21, maxiter 10, cancel 7, drift stop 1, not stated 42), the split of rounds by phase (27), the pressures (Table 2), and the proposed mechanisms (771 numbered suggestions); counts inside an issue are self-reports. A blind second LLM coder recoded a stratified sample of 30 postmortems mixed with 6 decoys and agreed with Cohen's $\kappa = 0.93$ on round counts, 0.82 on exits, and 0.79 on the primary pressure.

Table 2: **Review tails and convergence are the most common primary pressures.** Counts over the 118 postmortems; a report can show several pressures but has one primary pressure. κ: agreement with the second coder.

Pressure	Observed in the run	Present	Primary	κ
artifact state	tracker, summaries, contracts, lessons, review files	86	3	0.71
review tail	findings one at a time; piecemeal fixes; oscillation	80	36	0.77
scope and plan	size or ambiguity of the plan; scope creep; deferrals	79	16	0.82
convergence	no stop when due, or a stop without converging	76	27	0.70
evidence	environments, external jobs, test adequacy, sandbox	58	19	0.93
over-claiming	claims the evidence does not support	45	8	0.93
cost	tokens, context, compute, or time beyond rounds	25	1	0.71
human escalation	decisions that needed a human but were not routed	24	3	1.00
runtime and tooling	CLIs, hooks, operating systems, setup	13	4	0.63
other		4	1	–